[illegible]

[illegible]

[illegible]

```
[illegible]
```

```
def check_sorted_browser_connections_draw():
    clear_screen()
    if hasattr(menu, "switch_sp_tab"):
        menu.switch_sp_tab("CONNECTIONS")
    menu.draw_menu_background()
    menu.draw_sorted_browser(-1, False, "")

    # Restore default tab.
    if hasattr(menu, "switch_sp_tab"):
        menu.switch_sp_tab("SORTED")

def check_notebook_inspection_draw():
    clear_screen()
    gi.draw_notebook_inspection()

def check_safe_keypad_draw():
    clear_screen()

    setattr(gi, "safe_keypad_active", True)
    setattr(gi, "safe_keypad_code", "")
    setattr(gi, "safe_keypad_target", "???")
    setattr(gi, "safe_keypad_callback", None)

    gi.draw_safe_keypad()

    setattr(gi, "safe_keypad_active", False)

def check_desk_note_draw():
    clear_screen()

    setattr(
        gi,
        "active_desk_note",
        "TEST NOTE\nThis is a dummy desk note.\nPress SPACE to close.",
    )

    gi.draw_desk_note()

    setattr(gi, "active_desk_note", None)

def check_cutscene_draw():
    clear_screen()

    if not hasattr(gi, "cutscene_shots"):
        raise RuntimeError("gi.cutscene_shots missing")

    shot = gi.cutscene_shots[0]

    gi.draw_cutscene_scene(config.screen, shot, 0)
    gi.draw_dialogue_box(
        config.screen,
        shot.get("speaker", "NARRATOR"),
        shot.get("text", ""),
        min(80, len(shot.get("text", ""))),
        len(shot.get("text", "")),
    )

def check_start_mission():
    gi.start_mission()

    if getattr(gi, "player", None) is None:
        raise RuntimeError("gi.start_mission() did not create player")

def check_game_update():
    gi.game_update(EmptyKeys())

def check_game_draw():
    clear_screen()

    fcx = getattr(gi, "camera_x", 0)
    fcy = getattr(gi, "camera_y", 0)

    gi.game_draw(fcx, fcy)

def check_walking_sim_start_update_draw():
    clear_screen()

    gi.start_walking_sim()
    gi.update_walking_sim(EmptyKeys())
    gi.draw_walking_sim()

def check_pause_draw():
    clear_screen()
    menu.draw_pause_menu()

def check_death_draw():
    clear_screen()

    if hasattr(menu, "reset_death_animation"):
        menu.reset_death_animation()

    menu.draw_death_screen()

def check_autofire_update_path():
    """Catches missing _spawn_autofire_fire import in mission.py."""
    gi.start_mission()
    setattr(gi, "player_bottles", 3)
    gi.handle_throw_bottle()

    for _ in range(30):
        gi.game_update(EmptyKeys())

def check_mission_draw_story_characters():
    """Catches missing draw_widow / draw_commander imports."""
    gi.start_mission()

    if getattr(gi, "widow_pos", None) is None:
        raise RuntimeError("widow_pos missing")

    wx, wy = gi.widow_pos
    gi.game_draw(wx - config.WIDTH // 2, wy - config.HEIGHT // 2)

def check_walking_dialogue_draw():
    """Catches missing draw_dialogue_box import in walking_sim.py."""
    gi.start_walking_sim()

    setattr(gi, "walk_dialogue_active", True)
    setattr(gi, "walk_dialogue_speaker", "SYSTEM")
    setattr(gi, "walk_dialogue_text", "Smoke test walking dialogue.")
    setattr(gi, "walk_dialogue_char_idx", 99)

    gi.draw_walking_sim()

    setattr(gi, "walk_dialogue_active", False)

def check_mission3_chair_break_cutscene():
    """Drives the Mission 3 chair-break cutscene end-to-end and asserts the
    in-cell state mutations fire at the crack shot, the cutscene clock
    returns to -1, and the objective updates. (Spec §8.)
    """
    import mission3
    from state import STATE

    # Start Mission 3, then jump straight into the breakout: the chair-break
    # cutscene is gated on STAGE_BREAKOUT + mission3_chair_break_anim == 0.
    gi.start_mission3()
    STATE.mission3_stage = mission3.STAGE_BREAKOUT
    STATE.mission3_chair_broken = False
    STATE.player_disarmed = True
    STATE.mission3_weapon = "none"
    STATE.mission3_chair_break_anim = 0

    cb_end = mission3._CB_END
    crack = mission3._CB_CRACK

    # Run enough frames to clear the whole cutscene. We cap at cb_end + a
    # generous margin so a logic bug (counter never reaching _CB_END) surfaces
    # instead of looping forever.
    for _ in range(cb_end + 60):
        gi.game_update(EmptyKeys())
        if getattr(STATE, "mission3_chair_break_anim", -1) < 0:
            break

    # State mutations flipped at the crack shot must hold after the cutscene.
    assert getattr(STATE, "mission3_chair_broken", False), \
        "mission3_chair_broken did not flip True during the chair-break cutscene"
    assert not getattr(STATE, "player_disarmed", True), \
        "player_disarmed was not cleared during the chair-break cutscene"
    assert getattr(STATE, "mission3_weapon", "none") == "chair_leg", \
        "mission3_weapon was not granted as 'chair_leg' during the cutscene"
    # Cutscene clock returned to -1 (movement unlocks) and objective updated.
    assert getattr(STATE, "mission3_chair_break_anim", -1) < 0, \
        "chair-break cutscene clock did not return to -1 at _CB_END"
    assert mission3._objective == \
        "Sharpen the chair leg at the stone. Unlock/break the door.", \
        "objective text was not updated after the chair-break cutscene"

    # Sanity: the crack boundary used by the dispatcher is the canonical
    # cutscenes.CB_CRACK value — keeps the art/timing and the logic in sync.
    import cutscenes
    assert crack == cutscenes.CB_CRACK

def check_mission3_chair_break_shots_render():
    """Each cutscenes._draw_cb_* shot must render without raising at a
    representative 't' in its shot's range, and game_draw must take the
    cutscene branch without crashing (covers the dispatcher + the two
    cross-dissolves). (Spec §8.)
    """
    import cutscenes
    import mission3
    from state import STATE

    # Build a stand-in cell buffer the shots can composite onto. It mimics
    # the real cell's dominant colors (concrete wall / floor / metal chair)
    # so the readability checks below can confirm the shots ARE the cell
    # palette rather than an inverted one.
    W, H = config.WIDTH, config.HEIGHT
    cell_surf = pygame.Surface((W, H))
    cell_surf.fill((54, 50, 56))                        # concrete wall
    pygame.draw.rect(cell_surf, (34, 31, 33),          # floor
                     (0, H // 2, W, H - H // 2))
    # Metal chair back at center (seat_col / seat_dark).
    pygame.draw.rect(cell_surf, (58, 56, 60),
                     (W // 2 - 30, H // 2 - 5, 60, 40))
    pygame.draw.rect(cell_surf, (34, 40, 36),
                     (W // 2 - 34, H // 2 - 7, 68, 40))
    cx, cy = W // 2, H // 2

    # Direct shot draws at representative per-shot frame counters.
    def run_shot(fn, t):
        clear_screen()
        fn(config.screen, t, cell_surf, cx, cy)

    run_shot(cutscenes._draw_cb_strain, 0)
    run_shot(cutscenes._draw_cb_strain, cutscenes.CB_SHOTS_DUR // 2)
    run_shot(cutscenes._draw_cb_strain, cutscenes.CB_SHOTS_DUR - 1)

    run_shot(cutscenes._draw_cb_crack, 0)          # frame-0 white flash
    run_shot(cutscenes._draw_cb_crack, 5)
    run_shot(cutscenes._draw_cb_crack, cutscenes.CB_SHOTS_DUR - 1)

    run_shot(cutscenes._draw_cb_grab, 0)
    run_shot(cutscenes._draw_cb_grab, cutscenes.CB_SHOTS_DUR // 2)
    run_shot(cutscenes._draw_cb_grab, cutscenes.CB_SHOTS_DUR - 1)

    run_shot(cutscenes._draw_cb_ready, 0)
    run_shot(cutscenes._draw_cb_ready, cutscenes.CB_SHOTS_DUR // 2)
    run_shot(cutscenes._draw_cb_ready, cutscenes.CB_SHOTS_DUR - 1)  # final fade

    # Shared scaffolding helpers.
    cutscenes._cb_letterbox(config.screen)
    sp = cutscenes._cb_flame(config.screen, 50, 50)
    assert 0.0 <= sp <= 1.0

    # Readability check: the crack shot at full (unfaded phase) at peak
    # (t=wash) must actually contain the metal/splinter palette (proves the
    # shot matches the cell, not an inverted wood palette) and have
    # subject-region contrast (std-dev > 40) so it isn't a flat blob.
    import statistics as _st
    run_shot(cutscenes._draw_cb_crack, 10)
    px = pygame.surfarray.array3d(config.screen)
    region = px[cx - 60:cx + 60, cy - 60:cy + 60]  # chair-center 120x120
    flat = region.reshape(-1, 3).tolist()
    present_colors = set()
    for r, g, b in flat[::7]:
        present_colors.add((r, g, b))
    # Look for any pixel within ±24 of the metal / splinter / edge colors.
    def near(col, target, tol=24):
        return all(abs(col[i] - target[i]) <= tol for i in range(3))
    metal_targets = [cutscenes.CB_METAL, cutscenes.CB_METAL_DARK,
                     cutscenes.CB_METAL_EDGE, cutscenes.CB_SPLINTER,
                     cutscenes.CB_SPLINTER_HI]
    has_metal = any(any(near(c, m) for c in present_colors) for m in metal_targets)
    assert has_metal, \
        "crack shot region does not contain the cell's metal/splinter palette"
    lum = [(r + g + b) // 3 for r, g, b in flat]
    sd = _st.pstdev(lum)
    assert sd > 8, f"crack shot subject region too flat (std-dev={sd:.1f}), need >8"

    # game_draw cutscene branch: drive the clock across every shot boundary
    # (incl. the two cross-dissolve window) and the final frame.
    gi.start_mission3()
    STATE.mission3_stage = mission3.STAGE_BREAKOUT
    STATE.mission3_chair_broken = False
    STATE.player_disarmed = True
    STATE.mission3_weapon = "none"
    # Sample one frame per shot + each transition boundary.
    sample_frames = [
        0,
        mission3._CB_SHOTS_DUR // 2,
        mission3._CB_CRACK - 1,       # last strain frame
        mission3._CB_CRACK,           # crack, frame 0 (flash + mutations)
        mission3._CB_CRACK + 5,
        mission3._CB_GRAB - 1,        # last crack frame
        mission3._CB_GRAB,            # grab start
        mission3._CB_READY - 1,       # last grab frame
        mission3._CB_READY,           # ready start (cross-dissolve frame 0)
        mission3._CB_READY + 6,       # mid cross-dissolve
        mission3._CB_END - 1,         # final fade-out frame
    ]
    fcx = getattr(gi, "camera_x", 0)
    fcy = getattr(gi, "camera_y", 0)
    for f in sample_frames:
        STATE.mission3_chair_break_anim = f
        gi.game_draw(fcx, fcy)
        pygame.event.pump()
        pygame.display.flip()
    # Restore quiescent state.
    STATE.mission3_chair_break_anim = -1

def check_game_core_facade_state():
    """Checks that game_core still exposes old global-style names.

    This is especially useful if you use the proxy facade approach.
    """
    setattr(gi, "show_minimap", True)

    if getattr(gi, "show_minimap") is not True:
        raise RuntimeError("gi.show_minimap did not round-trip correctly")

    setattr(gi, "show_minimap", False)

    if getattr(gi, "show_minimap") is not False:
        raise RuntimeError("gi.show_minimap write failed")

def main():
    assert config.screen is not None, "config.screen was not initialised"

    ok = True

    print("\n=== Import checks ===")
    for module_name in GAME_MODULES:
        ok = run_import_check(module_name) and ok

    print("\n=== UI draw checks ===")
    ui_checks = [
        ("draw_menu", check_menu_draw),
        ("draw_settings", check_settings_draw),
        ("draw_sorted_browser_network", check_sorted_browser_network_draw),
        ("draw_sorted_browser_connections", check_sorted_browser_connections_draw),
        ("draw_notebook_inspection", check_notebook_inspection_draw),
        ("draw_safe_keypad", check_safe_keypad_draw),
        ("draw_desk_note", check_desk_note_draw),
        ("draw_cutscene", check_cutscene_draw),
        ("draw_pause_menu", check_pause_draw),
        ("draw_death_screen", check_death_draw),
    ]

    for name, fn in ui_checks:
        ok = run_check(name, fn) and ok

    print("\n=== Mission checks ===")
    mission_checks = [
        ("game_core_facade_state", check_game_core_facade_state),
        ("start_mission", check_start_mission),
        ("game_update", check_game_update),
        ("game_draw", check_game_draw),
    ]

    for name, fn in mission_checks:
        ok = run_check(name, fn) and ok

    print("\n=== Walking sim checks ===")
    walking_checks = [
        ("walking_sim_start_update_draw", check_walking_sim_start_update_draw),
    ]

    for name, fn in walking_checks:
        ok = run_check(name, fn) and ok

    print("\n=== Hidden branch checks ===")
    hidden_checks = [
        ("autofire_update_path", check_autofire_update_path),
        ("mission_draw_story_characters", check_mission_draw_story_characters),
        ("walking_dialogue_draw", check_walking_dialogue_draw),
    ]

    for name, fn in hidden_checks:
        ok = run_check(name, fn) and ok

    print("\n=== Mission 3 chair-break cutscene checks ===")
    chair_break_checks = [
        ("mission3_chair_break_cutscene", check_mission3_chair_break_cutscene),
        ("mission3_chair_break_shots_render", check_mission3_chair_break_shots_render),
    ]

    for name, fn in chair_break_checks:
        ok = run_check(name, fn) and ok

    for name, fn in hidden_checks:
        ok = run_check(name, fn) and ok

    pygame.quit()

    if ok:
        print("\nSMOKE TEST: OK")
        raise SystemExit(0)

    print("\nSMOKE TEST: FAILED")
    raise SystemExit(1)

if __name__ == "__main__":
    main()
```

```
----

"""art_cache.py — Centralised procedural surface cache for Dark of Distortion.

This module procedurally surfaces once and returns cached ``pygame.Surface``
objects via getter functions. This eliminates the repeated per-frame
``pygame.Surface()`` + ``pygame.draw.*`` creation found across mission.py,
mission2.py, mission3.py, ui_hud.py, weapons.py, death_anim.py, and
game_renderer.py.

All cached surfaces call ``.convert_alpha()`` for fast blitting (guarded
against the case where the display isn't initialised yet). Surfaces are
built lazily on first request and stored in module-level dict caches,
matching the ``_scaled_pause_cache`` pattern from config.py.

Usage example::

    from art_cache import get_blood_decal, get_panel_bg

    # Instead of creating a new Surface every frame:
    #   d_surf = pygame.Surface((r*2, r*2), pygame.SRCALPHA)
    #   pygame.draw.circle(d_surf, (100,10,10,255), (r, r), r)
    #   d_surf.set_alpha(int(decal_alpha))
    #   screen.blit(d_surf, ...)
    # Just do:
    decal = get_blood_decal(r)
    decal.set_alpha(int(decal_alpha))
    screen.blit(decal, ...)

After a display-mode change (e.g. toggling fullscreen), call
``rebuild_caches()`` to discard stale surfaces that were converted against
the old pixel format.
"""

import math
import random

import pygame
from config import *  # WIDTH, HEIGHT, font_body, C_* colors, font_* objects

# ── Cache registries ──

_blood_decals = {}
_panels = {}
_labels = {}
_glows = {}
_overlays = {}
_shadows = {}
_fire_scorch = {}
_fire_glow = {}
_vignettes = {}
_flicker_bars = {}

def rebuild_caches():
    """Flush all surface caches.

    Call this after a display-mode change (fullscreen toggle, resolution
    change, etc.) so that surfaces are re-built and ``.convert_alpha()``
    against the new pixel format. Without this, cached surfaces created
    against the old display format can cause slow blits or visual glitches.
    """
    _blood_decals.clear()
    _panels.clear()
    _labels.clear()
    _glows.clear()
    _overlays.clear()
    _shadows.clear()
    _fire_scorch.clear()
    _fire_glow.clear()
    _vignettes.clear()
    _flicker_bars.clear()

def _safe_convert_alpha(surf):
    """``.convert_alpha()`` that gracefully handles no display yet.

    config.py builds surfaces at import time assuming the display is ready
    (it calls ``pygame.init()`` and ``set_mode()`` before any factory runs).
    This guard mirrors that assumption but provides a fallback so that
    importing art_cache before display init doesn't crash.
    """
    if pygame.display.get_init():
        return surf.convert_alpha()
    return surf

# ── Blood decals ──

def get_blood_decal(radius):
    """Return a cached blood decal surface for the given radius.

    Builds a filled dark-red circle on a transparent ``(radius*2)`` square,
    matching the blood-decal style used across mission.py, mission2.py,
    mission3.py, and game_renderer.py.

    Callers apply per-frame alpha fade via ``surf.set_alpha(int(decal_alpha))``.
    The cached surface is built at full alpha (255) so ``set_alpha()`` maps
    directly to the original per-pixel alpha value.
    """
    if radius not in _blood_decals:
        surf = pygame.Surface((radius * 2, radius * 2), pygame.SRCALPHA)
        pygame.draw.circle(surf, (100, 10, 10, 255), (radius, radius), radius)
        _blood_decals[radius] = _safe_convert_alpha(surf)
    return _blood_decals[radius]

# ── HUD panel backgrounds ──

def get_panel_bg(w, h, alpha=160, border_radius=6, border_col=(60, 60, 60)):
    """Return a cached HUD panel background surface.

    Matches the panel styling used across mission.py/HUD-Intel and
    mission3.py/HUD-HUD: a semi-transparent black fill with a thin
    1-pixel border and rounded corners.
    """
    key = (w, h, alpha, border_radius, border_col)
    if key not in _panels:
        surf = pygame.Surface((w, h), pygame.SRCALPHA)
        pygame.draw.rect(surf, (0, 0, 0, alpha), (0, 0, w, h),
                         border_radius=border_radius)
        pygame.draw.rect(surf, border_col, (0, 0, w, h), 1,
                         border_radius=border_radius)
        _panels[key] = _safe_convert_alpha(surf)
    return _panels[key]

# ── Text labels ──

def get_label(text, font, color):
    """Return a cached rendered text label.

    Calls ``font.render(text, True, color)`` once and caches the result.
    Avoids per-frame ``font.render`` for common strings like "AMMO", "HP",
    "x3", "OBJECTIVE:", "TIMER", "MAP MODE", "[E]".

    The cache key uses ``id(font)`` because Font objects are not hashable.
    """
    key = (text, id(font), color)
    if key not in _labels:
        surf = font.render(text, True, color)
        _labels[key] = _safe_convert_alpha(surf)
    return _labels[key]

# ── Radial glow ──

def get_glow(radius, color, alpha=80):
    """Return a cached radial glow surface.

    Builds a soft radial gradient by drawing concentric circles with
    increasing alpha toward the center, matching the glow style used for
    muzzle flashes (mission.py/1569-1573) and pickup glows
    (mission.py/1886-1887).

    Callers can apply ``set_alpha()`` for per-frame pulsing.
    """
    key = (radius, color, alpha)
    if key not in _glows:
        size = radius * 2
        surf = pygame.Surface((size, size), pygame.SRCALPHA)
        cx = cy = radius
        # Draw concentric circles from the outside in, increasing alpha
        # toward the center (quadratic falloff — matches muzzle flash style
        # from config.py/105-107).
        for r in range(radius, 0, -2):
            t = 1.0 - (r / max(radius, 1))
            a = int(alpha * t * t)
            if a > 0:
                pygame.draw.circle(surf, (*color[:3], min(255, a)),
                                   (cx, cy), r)
        _glows[key] = _safe_convert_alpha(surf)
    return _glows[key]

# ── Full-screen overlays ──

def get_fullscreen_overlay(color):
    """Return a cached full-screen overlay surface of the given color.

    Builds a ``(WIDTH, HEIGHT)`` SRCALPHA surface filled with ``color``.
    Callers apply fade via ``set_alpha()``. Replaces per-frame
    ``pygame.Surface((WIDTH, HEIGHT), SRCALPHA)`` creation for hit flash
    (game_renderer.py/164-166), mission.py/1753-1763, finisher grayout
    (mission.py/1508-1510), and breath finisher overlays.
    """
    if color not in _overlays:
        surf = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        surf.fill(color)
        _overlays[color] = _safe_convert_alpha(surf)
    return _overlays[color]

# ── Fire zone surfaces ──
```

[illegible]